Università degli Studi di Milano
Facoltà di Scienze e Tecnologie
Corso di Laurea Magistrale in Informatica

REPRODUCTION AND ANALYSIS OF REAL-WORLD USE-AFTER-FREE ATTACKS AND STUDY OF DEFENCE TECHNIQUES

Relatore: Prof. Andrea LANZI

Correlatore: Dr. Andrea MONZANI

Tesi di:
Stefano NAVA
Matricola: 123456

Anno Accademico 2025-2026

Contents

List of Figures

Chapter 1

Introduction

Software security is nowadays a fundamental aspect of the development and maintenance of IT systems. The increasing complexity of applications, combined with the growing interconnection between devices and services, has significantly expanded the attack surface that can be exploited by potential malicious actors. Operating systems, browsers, drivers and distributed services are critical components whose proper functioning is closely linked to the robustness of their security properties.

In this context, proper memory management plays a central role, particularly in systems developed using low-level languages such as C and C++, which are defined as non-memory-safe. Although these languages offer high performance and control over resources, they are particularly prone to memory management errors, which can result in exploitable vulnerabilities. Memory-safe languages, in fact, have automatic mechanisms for controlling the allocation and deallocation of memory blocks. In the absence of these, while memory access is faster, an important safeguard for security is lost.

Despite decades of research and the development of numerous mitigation techniques, such vulnerabilities continue to arise even in widely used and mature software, highlighting how memory security remains an ongoing challenge in the contemporary cybersecurity landscape.

Chapter 2

Background

2.1 Memory structure

Memory is one of the most critical components from a security point of view in a computer system, since during program execution it contains not only the data being processed but also the information necessary to determine the program's behavior. This includes:

- **variables** containing temporary data used by the executed program(s);
- **data structures**, relationships between different types of objects for the purpose of managing more complex data;
- **pointers with memory addresses** which tell the program "where" a certain resource is. They can point:
  - to other objects, to allow the program to read or write data;
  - to functions, to allow the program to execute instructions located at a specific position;
- **information used to manage the execution flow**, such as the address of the first instruction that needs to be executed when returning from a function call.

Unauthorized access to or arbitrary modification of these elements can therefore have consequences ranging from simple data corruption or program crashes to changes in the control flow and, in the most severe cases, the execution of unauthorized operations, which is the subject we're going to examine. To understand the origin and effects of the main memory corruption vulnerabilities, it is therefore useful to first consider how the memory space associated with a program is organized into its main regions.

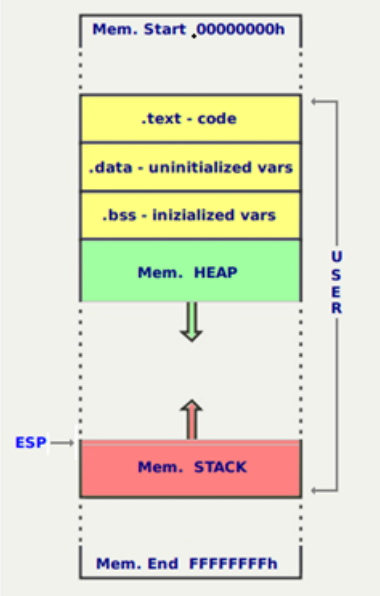

Figure 1: Structure of memory areas.

The two areas primarily affected by attacks of this kind are the stack and the heap.

2.1.1 Stack

The stack is a data structure organised according to a LIFO (Last In, First Out) policy and used primarily for managing function calls. Each function call involves the creation of a *stack frame* containing local variables, parameters and return information (such as the return address, which tells the program the point in the calling code from which to resume execution once the function has been exited). The allocation and deallocation of memory on the stack are automatic and handled by the compiler: when a function terminates, its corresponding frame is removed.

2.1.2 Heap

The heap is a memory region used for dynamic allocations, managed explicitly by the programmer using primitives such as `malloc`/`free` in C or `new`/`delete` in C++. These functions do not normally request a new memory region from the operating system for every individual location, but a user-space memory allocator manages larger regions of memory, dividing them into smaller blocks and keeping track of which portions are currently allocated or available for reuse.

On Linux systems, the allocator can obtain additional virtual memory from the kernel through mechanisms such as `brk()`, which changes the end of the process data segment, or `mmap()`, which creates new memory mappings. Large allocations are commonly serviced through `mmap()` while other allocations may be satisfied from memory already managed by the allocator or from an enlarged

process heap. The exact strategy is implementation-dependent and may change according to allocation size and configuration. [19] [20]

Unlike the stack, therefore, heap allocations do not follow a strict LIFO (Last In, First Out) order: allocated blocks may be released independently and subsequently reused for other object, which is essential for dynamically sized and long-lived data structures, but it also makes object lifetime and memory reuse more difficult to manage correctly, creating the conditions in which errors like UAF may appear.

2.1.3 Pointers

In memory-unsafe languages, pointers are identifiers that may contain the address of a specific memory area (to which they "point") and, for example in C, are initialised using the `malloc` function and freed using the `free` function. The programmer can use them to perform operations both on the addresses they contain and on the values contained in the memory cells they point to.

Given their nature, they are one of the main vectors for memory-related attacks. We therefore introduce the concept of a **dangling pointer**: a pointer, if manipulated incorrectly, may continue to point to an area that is no longer valid, typically because it has been deallocated or has expired. This allows an attacker to read or write content to which they should not have access.

Listing 2.1: Example of dangling pointer in C.

```c
void dangling_pointer_example(void){
    int *ptr = (int *)malloc(sizeof(int));
    *ptr = 5;
    free(ptr); // ptr is now a dangling pointer
    ptr = NULL; // now it doesn't point to the freed zone anymore
}
```

Under the right conditions, exploiting a dangling pointer can lead to **privilege escalation**, i.e. a low-level user gaining administrator privileges, thereby putting sensitive data or, in the most serious cases, entire systems at risk.

2.1.4 Reference counting mechanisms

To mitigate the problem of dangling pointers and to avoid placing the responsibility for memory management directly on the programmer, **reference counting** mechanisms have been introduced. These mechanisms associate each object with a counter that tracks the number of active references to it, so that, provided there are no errors, the relevant memory area is freed when it is no longer in use.

Listing 2.2: Simple example of a reference counting mechanism.

```c
void reference_count_example(void) {
    int *ptr = malloc(sizeof(int));
    int ref_count = 1;       // starting with 1 existing reference

    ref_count++;             // new reference
    ref_count--;             // release of a reference
    ref_count--;             // release of the last reference

    if (ref_count == 0)
        free(ptr);
}
```

When a new structure gains access to the object, the counter is incremented. Conversely, when the reference is no longer needed, the counter is decremented. When the counter reaches the value *zero*, the system considers the object to be no longer in use and frees up the memory it occupies.

This approach helps to avoid premature deallocations and to coordinate concurrent access to shared resources more reliably. However, errors in counter management (such as multiple decrements or missing increments) can lead to critical conditions.

Early deallocation of an object that is still in use can, in fact, result in *dangling pointers* and use-after-free vulnerabilities, while failure to release references can cause memory leaks and uncontrolled memory consumption. For this reason, reference counting is one of the most delicate mechanisms in memory management within memory-unsafe systems.

2.2 Use-after-free (UAF)

2.2.1 What causes a UAF?

Every system has flaws and is vulnerable to attacks of varying severity. These flaws, which often stem from logical errors or the incorrect management of the lifecycle of objects in memory, can enable an attacker to compromise the integrity of the system, with different objectives. Some of the most known are **buffer overflow** (a process uses a memory area that hasn't been allocated to it) or **double free** (a memory area is deallocated two times corrupting the structures of the memory allocator).

The type of error we're going to focus on is **use-after-free**. Incorrect management of the dynamic memory lifecycle can lead to access to portions of memory that have already been deallocated, allowing an attacker to reuse these areas to inject controlled data and influence the program's behaviour via dangling pointers.

Listing 2.3: Simple example of a UAF exploit in C.

```c
void uaf_example(void) {
    int *ptr = (int *)malloc(sizeof(int));
    *ptr = 5;

    free(ptr);        // ptr is now a dangling pointer

    *ptr = 10;        // use-after-free: access to freed memory

    ptr = NULL;       // the pointer is "neutralized"
}
```

A use-after-free vulnerability is fundamentally a violation of the expected lifetime of an object, which passes through three conceptual phases: allocation, usage and deallocation. During the usage phase, one or more parts of the program can hold references to the object and assume that the memory area remains valid. Once the object is deallocated, this isn't true anymore and every reference that could still be used (*dangling pointers*, as explained in section 2.1.3) should be removed to avoid producing use-after-free conditions.

It's important to distinguish the lifetime of an *object* from the one of a *pointer* referring to it. When the `free()` function is called, the allocator marks the memory region as available for future use, but it doesn't remove the existing references to it, which could still be used. In large systems, such as the Linux kernel, this distinction is even more important because references to the same object may be interacted with in different execution contexts.

As mentioned in section 2.1.4, a solution to this problem is counting references for the objects, but an early deallocation can result in a dangling pointer and can therefore be a specific (but not the only) cause of a use-after-free bug (and a potential exploit as a result).

2.2.2 Exploiting UAF and refcount errors

From an exploit perspective, the dangling reference resulting from this lifetime inconsistency represents only the initial condition of the attack. The object may have already been freed while an execution context still retains its address. The following example illustrates how one can read a value in a freed memory zone exploiting a dangling pointer.

Listing 2.4: Example of a read operation exploiting a UAF caused by a refcount error.

```c
typedef struct {
    int value;
    int refcount;
} Object;

void uaf_refcount(void) {
    Object *obj = malloc(sizeof(Object));
    obj->value = 5;
    obj->refcount = 1;

    Object *ptr = obj;       // new reference
    /* BUG: missing obj->refcount++ */

    if (--obj->refcount == 0)
        free(obj);           // obj is freed early

    int value = ptr->value;  // use-after-free
}
```

In more complex (and realistic) cases, exploitation relies on turning this stale reference into a predictable and controllable interaction with the allocator and with subsequent allocations.

The first stage of an exploit generally involves forcing an incorrect transition in the lifecycle. Depending on the vulnerability, the attacker may need to trigger an error-handling path, repeatedly register and unregister a resource, close an object while it is still in use by another operation, or coordinate multiple concurrent operations. In vulnerabilities relating to reference counting, the key event is often the premature release of the last reference recognised by the system, even though another reachable reference still exists. The attacker must therefore maintain a code path capable of accessing the object after it has been deallocated, while simultaneously causing the counter to reach zero.

This phase can be particularly complex in concurrent systems. One thread might acquire or retain a pointer to an object, while another thread removes the object from a shared structure and frees what the system considers to be the last reference. If the acquisition of the reference is not correctly synchronised with the destruction of the object, we could end having a deallocation between the retrieval of the pointer and the increment of the counter. Similarly, a callback or an asynchronous task may retain an address without actually holding a reference to it. An attacker could attempt to widen these race condition windows by increasing the system's load or using multiple threads.

Once the object has been freed, the dangling pointer alone gives us only limited control. The next objective is to trick the allocator into reusing the freed memory for a new allocation whose contents can be controlled by the attacker (= **heap**

grooming or **heap shaping**). The attacker performs a controlled sequence of allocations and deallocations to manipulate the state of the heap and increase the likelihood that a chosen object will occupy the same memory region as the vulnerable object.

The feasibility of this replacement depends heavily on the behaviour of the allocator. Allocators, whether in kernel space or user space, tend to group allocations into size classes and to reuse freed regions for compatible objects. Consequently, the attacker typically seeks a replacement object belonging to the same allocation class as the vulnerable structure and which can be created repeatedly via an accessible interface.

Following memory reuse, two logically distinct objects occupy the same region at different times. The vulnerable code still interprets the dangling pointer according to the layout and semantics of the original object (as it should), while the allocator treats the region as belonging to the new object. This creates a form of type confusion induced by memory reuse: the fields of the new object may be interpreted as flags, sizes, reference counters, pointers to lists or function pointers belonging to the original structure.

The resulting access can provide various exploit primitives. A read via the dangling pointer may reveal heap addresses, kernel pointers or other sensitive values, making it easier to bypass mechanisms such as ASLR (Address Space Layout Randomization: a technique for randomizing addresses of stack, heap and libraries in memory which usually helps preventing memory attacks). If a corrupted field is subsequently used as a pointer or index, we can obtain out-of-bounds read or arbitrary memory access primitives. A write via the obsolete reference, on the other hand, can alter the state of the new object, resulting in a controlled or partially controlled write to another area of memory.

Objects that contain pointers to functions or references to related structures are of particular interest to us, as their corruption can indirectly affect the control flow. However, modern exploits do not necessarily aim to immediately overwrite a function pointer. Mechanisms such as Control-Flow Integrity (CFI), memory non-executability and restrictions on code execution in kernel space often make direct redirection difficult. Instead, the exploit uses the initial vulnerability to construct more powerful read and write primitives, obtain address information and modify sensitive data structures. These primitives can then be exploited to alter credentials, permissions or the state of a process, depending on the execution context.

A pending reference may also interact directly with the reference counting mechanism. If the obsolete pointer is used in a deallocation operation, the system may decrement a field that no longer represents the original counter. This may corrupt the new object or cause a second deallocation of the same memory region. In such cases, the initial error may lead to additional conditions such as double-free or premature destruction of the new object. The exploit may therefore involve multiple consecutive violations of the object's lifecycle.

This entire exploitation chain can be represented as a sequence of steps:

1. Triggering an incorrect reference-counting transition

2. Maintaining a reachable dangling pointer
3. Reusing the freed memory with a suitable replacement object
4. Accessing the new contents via the obsolete type
5. Transforming the resulting behaviour into a useful (usually malicious) memory primitive

Although this is only a generalized explanation of the idea and process behind exploiting this type of vulnerability, it usually isn't easy as it seems, due to the need for certain conditions to be met.

2.3 Linux and its kernel

2.3.1 The kernel

The kernel is the central component of a Linux operating system and operates with higher privileges than normal programs running in user space. Its main task is to mediate access to hardware resources and to provide applications with a controlled set of services via system calls. This is convenient for the developer, but it also carries some risks: while a bug in a user space program is confined to its process, memory corruption in kernel space can compromise the stability of the entire system or allow operations to be carried out with elevated privileges (the *privilege escalation* that we mentioned in section 2.1.3).

In addition to this, Linux keeps a highly modular structure, as some functionality can be compiled as a loadable module. However, this flexibility does not provide complete isolation: a loaded module continues to run in kernel space and can access the kernel's internal structures, with the same aforementioned security implications.

Since its first release in 1991, Linux has evolved from a small-scale project into a platform used on a vast number of servers and devices all around the world, as we can see on the following table which sums up the distribution of Linux-powered machines around the world [1]:

Metric	**Linux Share**
Server OS Market (2026 est.)	51.3%
Websites (known OS)	61.3%
Top 1 Million Web Servers	96.3%
Government Data Centers	64.9%
Top500 Supercomputers	100%

This evolution has been accompanied by the introduction of numerous hardware architectures, new drivers and subsystems, which support a broader quantity of

features but bring more security and stability issues on the kernel's side. The interfaces exposed to user space are kept as stable as possible, while the kernel's internal APIs may change significantly between different versions.

2.3.2 Use-after-free vulnerabilities in the Linux kernel

Use-after-free vulnerabilities are particularly relevant in the Linux kernel because many kernel objects are dynamically allocated, shared among different subsystems and accessed concurrently by multiple execution contexts (e.g. sockets, network objects, device structures). Their lifetime, as described in section 2.1.4, is often managed through mechanisms like reference counting which, in case of wrong manipulation, can produce dangling pointers and subsequent use-after-free exploits.

As stated previously, the consequences of errors like these are generally more severe than in a normal user-space program, since kernel code executes with high privileges. Detecting these vulnerabilities is complicated by the structure of the kernel itself. The allocation, use and release of an object may occur in different context and the relevant sequence of operation is likely not visible within a single control-flow path (this includes the thread concurrency problem seen in section 2.2.2).

These characteristics make automated kernel analysis particularly complex. A detection tool has to deal with code spread across multiple modules that interact in a non-linear manner. Furthermore, some tools require a specific version of the kernel, the compiler or their dependencies, while others necessitate changes to the build system or a full instrumentation phase. The continuous evolution of internal APIs can quickly make unmaintained tools incompatible. The kernel documentation distinguishes between numerous categories of tools, such as:

- *static analyzers*: code analysis by tracking objects' paths without executing the code. An example is UAFX, which we tested and discuss in section 2.4.1
- *sanitizers*: suspect behaviour detection by instrumentating the code through a compiler
- *code coverage tools*: measurement of the quantity of code covered by tests
- *fuzzing tools*: generation of random inputs in order to test different execution paths for the analyzed software
- *mechanisms for detecting race conditions*: identification of critical code sections where a race condition may happen

LLVM

Most of the tools we have considered are based on LLVM. It is a modular infrastructure for building compilers and code analysis tools. Its central component is an intermediate representation (generated by a compiler; one of the most used is Clang), commonly referred to as LLVM IR, on which modifications can be made

that are independent of the source language and, to some extent, of the target architecture.

The use of LLVM is significant in the field of security because it allows analyses and transformations to be incorporated directly into the compilation process (e.g. by adding checks that will be executed at runtime). This integration enables more in-depth analyses than simply processing the source code textually, albeit at the expense of the performance of the system being analysed; however, a system compiled with LLM will likely not be made available to a user but will be used solely for analysis purposes.

2.4 Existing tools for detecting UAF vulnerabilities

For all known bugs, there is a wide range of tools described in the academic literature, used either individually or in combination, that serve (or help) to detect them. UAF issues, however, especially when related to reference counting problems, are particularly challenging. Due to the unpredictable nature of these bugs, the numerous conditions required to exploit them, and the variety of contexts in which they can occur, identifying them is by no means easy. In addition to the identification process itself, other problems may arise, such as large volumes of false positives that can cause a tool to be inaccurate on its own and require human review to distinguish them from true positives, or the need for extreme computational power to run the tool effectively.

Below are some examples of tools capable of analyzing code and detecting use-after-free vulnerabilities. Since each vulnerability exploits different weaknesses, we have compiled a list of tools with different purposes and modes of operation to provide a broader range of options in this regard.

Tool	LLVM-based	Kernel-usable?	Public source?
UAFX [2]	Yes	Yes	Yes
CountDown [3]	Yes	Yes	Yes
Infer [4]	Yes (+ Clang)	Yes	Yes
Clang Static Analyzer [5]	Yes	Yes	Yes
Syzkaller [6]	Yes	Yes	Yes
GCC-fanalyzer [7]	No	No	Yes
CRed [8]	Yes	No	No
Cppcheck [9]	Yes	No	Yes
SVF-derived tools [10]	Yes	No	Yes
FreeWill [11]	Yes	No	No
LinkRID [12]	Yes	Yes	No
CRCount [13]	Yes	No	No
UAFChecker [14]	Yes	No	No
DCUAF [15]	Yes	Yes	No
SinkFinder [16]	Yes	No	No
Goshawk [17]	Yes	Yes	Yes
CID [18]	Yes	Yes	No

There were two tools specifically designed to detect UAF bugs in the Linux kernel: CountDown and CID. Unfortunately, CID does not have publicly available source code, and, as with other tools that do not meet this criterion, we decided to exclude them from the selection.
Since our focus was on a use-after-free vulnerability involving a reference-counting issue, we also excluded some tools whose papers clearly stated that they would not be effective for this purpose.
Finally, we excluded the tools that would not have worked on the kernel because they were designed for smaller, less complex sections of code.
After considering these factors, the decision was made to choose UAFX and CountDown.

2.4.1 UAFX

UAFX is a static analyzer, presented at NDSS 2025 [2], designed to detect *cross-entry* (non-linear) use-after-free vulnerabilities in the Linux kernel. The term "cross-entry" refers to conditions in which the deallocation and subsequent use of an object occur through different entry points (e.g system calls, callbacks, driver

interfaces). In such cases, the link between the deallocation point and the usage point may involve parts distributed across distinct execution paths, making it difficult to detect using analyzers limited to a single function or a single call chain (i.e. where memory allocation, usage, and deallocation occur in a linear pattern). UAFX's approach is based on achieving two main macro-objectives:

- **Escape-Fetch-based Cross-Entry Alias Analysis**
- **Systematic Partial-Order Constraint-based UAF Validation**

We describe them with reference to the following figure:

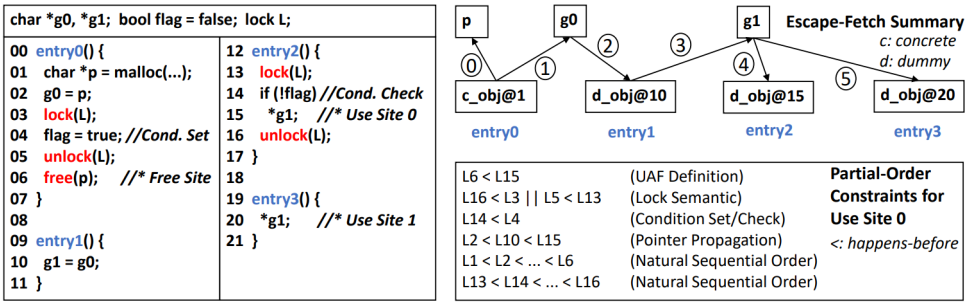

Figure 2: Example of code entry points and how they are handled by UAFX.

Escape-Fetch-based Cross-Entry Alias Analysis

UAFX treats object assignments to pointers as escapes and retrievals (from "unknown" areas) as fetches. UAFX attempts to construct Escape-Fetch Paths (EFPs) to define aliases that track memory allocation, usage, and deallocation from different access points. This is necessary because, from the perspective of a single function, an object may have an unknown origin (as is commonly the case with an externally initialized pointer, such as `g1` in `entry3`).

Systematic Partial-Order Constraint-based UAF Validation

Once the aliases/paths have been confirmed, UAFX verifies that the conditions for a UAF actually exist. If, for example, there is a check on a flag (`entry0`, `entry2`) that is manipulated by multiple entry functions and used to prevent a UAF on a critical pointer, that path must be discarded.

Workflow

We can break down UAFX's workflow as follows:

1. **Identifying entry functions.** This is done semi-automatically, starting with:
    - *General call analysis*: functions that serve as entry points typically have no callers in the program, so they are candidates.

- *Software-based heuristics*: the paper uses drivers as an example; these typically have entry points in a `file_operations` structure, which makes them easy to identify.

After these initial automated steps, a manual review is conducted to exclude unnecessary candidates or to add others.

2. **Code analysis and summary for each entry.** The entry functions are analyzed, and all escape/fetch objects, critical regions, and condition sets/checks (such as the flag in the example) are identified. UAFX generates a summary of all possible paths (in Figure 2, the graph in the upper right).

3. **Cross-entry detection of free and use sites with aliases.** UAFX considers all sites where memory regions are freed and used as potential candidates for UAF. In the example, with the path `entry0-entry3`, defined as follows (between the *free* operation on line 6 and the *use* operation on line 20):

$$(\text{c\_obj@1} \longrightarrow \text{g0} \longrightarrow \text{d\_obj@10} \longrightarrow \text{g1} \longrightarrow \text{d\_obj@20})$$

there is a potential UAF.

4. **UAF Validation.** UAFX considers all candidates and evaluates their constraints within the path (order, locks, set/check conditions) to determine whether each one is actually a vulnerability. It also verifies that the free and use sites actually access the same object, taking into account the entire escape-fetch path. For this purpose, SMT solvers can be used (the paper cites z3 as an example).

Thread tracking

Since some UAFs are caused by race conditions, UAFX recognizes two types of thread-related lifecycles:

- **Thread creation**: calls that create child threads (`pthread_create`, `fork`). UAFX retrieves information from these events (e.g., the thread's entry function or parameters)

- **Thread joins** (`pthread_join`, `wait`): events that ensure threads exit before a certain point in the parent thread. UAFX pairs each exit event with its corresponding creation event to track the entire thread lifecycle.

UAFX and CVE-2025-21756

Speaking specifically about our case, UAFX may not be able to automatically identify the UAF for CVE-2025-21756 because of two main issues.

A first obstacle could be identifying the entry functions. As described earlier, automatic identification of candidates occurs for functions that have no callers, but this is not the case here because the functions that operate on socket objects

are instead called by other functions. This could be resolved in the second step of the identification process, namely by manually adding the functions to the set of candidates.

Another, more significant obstacle is the difficulty explicitly described in tracking reference counts (page 9 of the paper [2], *Reference Count Mechanisms*). UAFX intentionally avoids analyzing candidates involving reference counts, which is, in fact, the main cause of CVE-2025-21756, because of the excessive number of false positives that would be generated, in order to maintain high precision (but lower recall). Despite this, it would be theoretically possible to extend UAFX with the help of other previouslky mentioned tools (here and in UAFX's paper): LinKRID (page 14), CRCount, and FreeWill (page 15) can, with some modifications, be used together with UAFX to increase accuracy on reference count problems.

As described in section , we tried making a slight modification to the UAFX code to see if we could also detect the reference counting issue.

2.4.2 CountDown

CountDown is a fuzzer for the Linux kernel, presented at CCS 2024 [3] and, unlike UAFX, it's specifically designed to detect memory timing errors caused by incorrect handling of reference counters. Unlike traditional fuzzers, which are primarily driven by code coverage, CountDown uses operations on reference counters as feedback to identify sequences of system calls that could potentially alter the lifecycle of those same objects. The tool is built on top of Syzkaller and extends its strategies for generating and mutating test programs.

During execution, CountDown instruments the kernel to associate increments, decrements, and accesses to reference counters with the system calls that triggered them. Based on this information, it establishes relationships between system calls that operate on the same counters and assigns them a higher probability of being combined in the generated tests. The tool can also repeatedly insert operations that decrement a counter, in an attempt to trigger the object's deallocation, followed by operations that continue to access it. In this way, fuzzing is directed toward sequences that can more effectively expose UAFs caused by incorrect operations on counters.

CountDown's approach is based on the idea of refcount-driven fuzzing to achieve two macro-objectives:

- *Identification of refcount-based relationships between syscalls*: instead of relying solely on syscall signatures or coverage, CountDown correlates syscalls that operate on the same object (*shared refcounts*).
- *Systematic exposure of UAF bugs*: through targeted mutations, the tool attempts to force an object's refcount to zero and subsequently access that object via syscalls that share its reference.

Workflow

CountDown's workflow can be broken down into three main phases:

1. **Kernel instrumentation and logging**: CountDown instruments the refcount APIs in the Linux kernel to log every refcount operation using a 4-element tuple:

$$< S, O, \delta, V >$$

where a syscall S updates the refcount O of a refcount by a value δ. V is the final value.

The instrumented APIs are listed in the following table:

Name	Description
`refcount_set`	set refcount to the given value
`refcount_add`	add the given value to refcount
`refcount_inc`	increase refcount by 1
`refcount_dec`	decrease refcount by 1
`refcount_add_not_zero`	add given value to refcount if refcount is not 0
`refcount_inc_not_zero`	increment if the refcount is not 0
`refcount_dec_not_one`	decrement if the refcount is not 1
`refcount_dec_if_one`	decrement if the refcount is 1
`refcount_dec_and_test`	decrement refcount and check if result is 0
`refcount_sub_and_test`	subtract value from refcount and check it with 0

To identify the same object across different virtual machines (VMs), it uses a checksum of the call stack at the time of the object's initialization through `refcount_set`.

CountDown creates a RAI map (`refcount-address-ID`) to link the addresses of the refcounts with their refcount IDs.
When a refcount is initialized, a new entry in the RAI map is created with its address and ID.
When a refcount reaches zero, the entry is removed from the map.
When a refcount is incremented or decremented, CountDown searches the address in the map to find its ID and records the operation.

During fuzzing, to connect refcount operations with syscalls, CountDown needs to record the Syzkaller syscall number (NR) for each operation. This is done in a thread-safe manner in order to avoid race condition, since the Linux kernel utilizes multiple threads to execute parallelized syscalls.

While tracking the refcount values, CountDown considers that one syscall may modify a refcount many times, but it only cares about the final variation (δ). In order to do so, every refcount API listed in table 1 is modified

to record every single refcount change. This data is kept in a two-dimension table (one for the syscall number, one for the refcount ID), in which every entry records the current δ and the current V.

2. **Reworking relationships between system calls**: CountDown constructs multiple relations among syscalls and refcounts starting from the refcount operations mapped in the 4-tuple format (S, O, δ, V).

   First, it identifies syscalls that effectively change the refcount of any object. It remaps a set of 4-tuples in a new map, where the key is the refcount ID and the value is a set of syscalls and their changes to the refcount (δ), avoiding relations between syscalls from different programs (which can quickly become unrealiable).

   Based on the shared refcounts, every relation is reworked in a new one defined as `RefcntRelation`. The assumption is that, if two syscalls access the same refcount within one execution, they have a stronger relation with respect to triggering more complicated refcount operations. Each syscall set associated with a specific refcount is then split into a set of syscalls pairs. Following that, CountDown creates the syscall relation table, where each unique pair of syscalls has an entry and the value is the number of shared refcounts accessed by both.

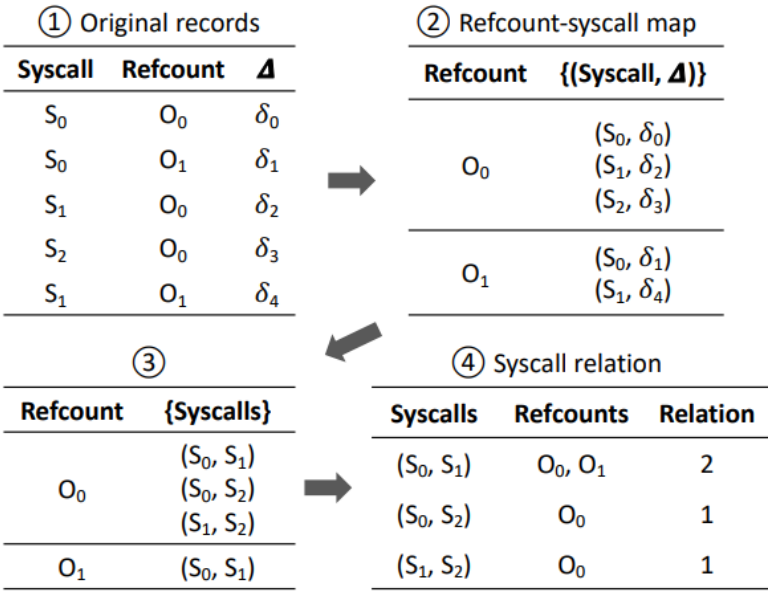

Figure 3: Example of refcount-based syscall relation construction.

As explained in the paper, this newly constructed relation is more efficient in detecting refcount-related UAF bugs. However, the kernel code base is very large and refcount operations are not densely distributed. To solve the problem, the `RefCntRelation` is merged with the Syzkaller's standard

`SyzRelation` and handled based on the fuzzing time. At the beginning of fuzzing, CountDown assigns a high priority to the original coverage-based relation to quickly explore more kernel code. As time goes on, it allocates more weight to CountDown's own relation so that the fuzzer can focus on testing complex refcount operations. This can be represented as the following formula:

$$OverallRelation = log_2(SyzRelation) + k * log_2(RefcntRelation)$$

k starts with a small value and is incremented every time after testing a fixed number of test cases.

Since fuzzers are non-deterministic, different kernel instances can gather different relations. When a new one is received from one VM, CountDown synchronizes it across all instances (to ensure that the other instances can directly make use of it) by fetching each VM's report, merging them and then distributing the result to all instances.

3. **Guided program mutation**: CountDown uses the learned relationships to generate new programs. It starts by using the mutators from previous fuzzing tools (like Syzkaller) to mutate the program. Having assigned higher weights to refcount-based relations, the mutation will prefer combining syscalls that are strongly related regarding refcount operations.

   Then it creates a new mutator that combines refcount-related syscalls more aggressively. CountDown chooses one target program and runs it in order to collect all accessed refcounts. It will update the refcount-syscall map if the execution triggers new records. To mutate the program, it will randomly select a refcount from the collected set. If a refcount was accessed multiple times, it will have a higher probability to be selected.

   CountDown then searches the map for syscalls that operate on this refcount and selects one for insertion into the program, which is done reusing Syzkaller's logic, preferring a later position, because the syscalls at the beginning may work on the initialization of resources and refcounts.

   In addition to new code coverage, CountDown considers any program that exhibits a new `(syscall, refcount)` pair never seen before to be "interesting" and saves it for future changes. This is done while fuzzing through checking whether the execution triggers new code coverage or a new refcount operation, which was previously missed. If that's the case, it saves the new program into the input queue for further mutations and updates the refcount-syscall map with the new entry.

   As a final but important point, a triggered refcount error will not immediately result in a memory error. CountDown needs to trigger more operations to make the kernel release the object and access it through dangling references. It does so by inserting more refcount-related syscalls into the executed program in order to reduce the gap between a refcount bug and a UAF, so that sanitizers like KASAN can capture it.

Limitations

Although CountDown is optimized to detect UAFs with refcounts, it still has some limitations:

- **Generic refcounts**: some refcounts (such as those in the AppArmor security module) are used by nearly all syscalls, creating "noise" and artificial relationships that CountDown must filter out to maintain accuracy.
- **Coverage trade-off**: because it focuses heavily on refcount logic, CountDown may record slightly lower total code coverage (approximately 1.8% – 4.3% less) than Syzkaller, as it ignores paths unrelated to object management.
- **Instrumentation overhead**: although optimized through selective logging (excluding interrupts and asynchronous tasks), the constant monitoring of every refcount operation introduces a higher computational load than traditional fuzzing.

Chapter 3

CVE-2025-21756

3.1 Overview

CVE-2025-21756 is a use-after-free vulnerability identified in the Linux kernel's AF_VSOCK subsystem, which is used for communication between hosts and virtual machines. The vulnerability allows a local attacker to gain elevated privileges (up to root) by exploiting incorrect management of the socket object lifecycle within the kernel.

The problem arises during specific sequences of operations on vsock sockets, leading to the dereferencing of memory structures that have already been freed and, consequently, to kernel memory corruption. Specifically, an attacker could exploit this vulnerability to achieve privilege escalation.

3.2 The VSOCK subsystem

VSOCK is a component of the Linux kernel that manages communication between a virtual machine (*guest*) and the system hosting it (*host*). It was developed in response to the need to provide a method of communication between the two entities that offers better performance than traditional network interfaces. Unlike TCP/IP sockets, which are based on IP addresses and network protocols, vsock sockets use an addressing model specific to virtualised environments. Communication via vsock utilises a pair of identifiers:

- **CID (Context IDentifier)**: uniquely identifies an endpoint (host or VM)
- **Port**: similar to TCP/UDP ports

Some relevant CID values may include:

- `VMADDR_CID_HOST`: represents the host
- `VMADDR_CID_LOCAL`: local communication
- specific CIDs assigned to individual VMs

The advantage of this model is that it avoids the overhead associated with the IP stack.
Communication takes place via two transport networks:

- **G2H (guest-to-host)**: these are executed on the guest and are usually device drivers. We distinguish between three types:
  - *virtio-transport*: typical of KVM/QEMU, it uses virtqueues located within the guest as buffers, but the host also reads from and writes to them.
  - *vmci-transport*: VMWare's proprietary for H2G/G2H.
  - *hyperv-transport*: based on Hyper-V VMBus, designed specifically for Microsoft environments.
- **H2G (host-to-guest)**: these are run on the host and usually provide device emulation for the guest. There are two types:
  - *vhost-transport*: host-side backend for virtio-transport.
  - *vmci-transport*: VMWare's proprietary for H2G/G2H.

To illustrate communication between the host and the guest, we can use QEMU, an open-source emulator and virtualisation program, as an example. When creating a virtual machine, the structure will be as follows:

- *virtio-transport* on the guest
- *vhost-transport* on the host: provides in-kernel device emulation to the guest and uses `ioeventfd` and `irqfd` to receive and send events to the guest.

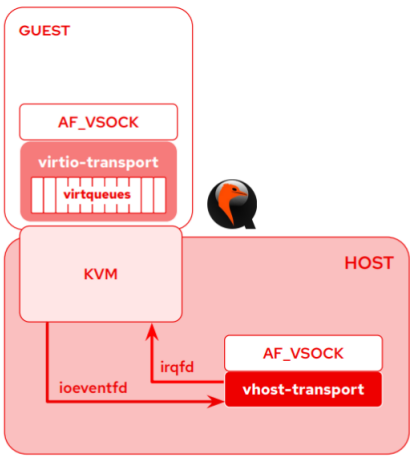

Figure 4: Example of a VSOCK communication structure.

Both provide an interface to the socket layer via the vsock core and exchange packets with virtio queues (*virtqueues*).
KVM (Kernel-based Virtual Machine) turns the host OS into the VM's hypervisor, enabling it to manage the CPU and memory directly.

Virtqueues

These are shared buffer queues in memory that allow the two parties to communicate bidirectionally (even though a single virtqueue is dedicated to a single direction) while maintaining low overhead.
In **split virtqueue** mode (classic), they are based on three main shared memory areas:

- **Descriptor Area**: contains the buffer metadata (the guest's physical address and length).
- **Available Ring**: a list of buffers that the driver has added and which are ready to be consumed by the host device.
- **Used Ring**: a list of buffers that the device has finished processing and is returning to the driver.

The **packed virtqueue** mode, on the other hand, uses a more modern and compact layout, merging the available and used rings, thereby improving the performance of the processor cache.
The operation can be broken down into the following stages:

1. **Insertion**: the VirtIO (guest) driver places the buffers in the *descriptor area* and updates the *available ring*.
2. **Notification (Kick)**: The driver sends a notification to the device ("doorbell") to indicate that new data is available.
3. **Processing** of data by the host.
4. **Completion**: the host updates the *used ring* and sends an interrupt to the guest to signal completion.

The virtqueue backend (which actually manages the queue on the host) can be **QEMU**, with full emulation, as in the case under consideration, or **vhost**: the backend runs within the host kernel (vhost-net/vhost-blk), allowing the driver to communicate directly with the kernel without going through QEMU's user space, thereby drastically reducing latency.

Internal architecture of VSOCK

From an implementation perspective, each vsock socket is represented in the kernel by a data structure (`struct vsock_sock`), which extends the generic socket structures (`struct sock`). These objects are managed via:

- **status lists**:
  - *bound sockets*: associated with a port;
  - *unbound sockets*: created but not yet associated;
- **connection queue**: handling of incoming requests;

- **reference counting**: a fundamental mechanism for managing the life cycle of objects.

When consistency in the management of these elements is lacking, there is a risk of encountering the problems mentioned above, exposing the system to risks and attacks.

The life cycle of a VSOCK socket

Typically, a vsock socket goes through the following stages:

1. **Creation** (`socket()`): memory is allocated for the data structure, which is then added to the *unbound* list.

2. **Binding** (`bind()` or autobind): the socket is associated with a port and is consequently added to the *bound* list.

3. **Connection** (`connect()` / `accept()`): the actual communication channel is established.

4. **Release** (`close()`): when the socket is no longer needed, it is removed from the internal structures, its reference count is decremented and, if this reaches zero, its memory is deallocated.

3.3 Test case presented by the developers

Let's start with a test case provided by the maintainers of the VSOCK module for the Linux kernel, who, after releasing the patch, made it available to allow users to check whether their system is affected by the issue.

Listing 3.1: Test case presented by the developers.

```
#define MAX_PORT_RETRIES        24
#define VMADDR_CID_NONEXISTING  42

/* Test attempts to trigger a transport release for an unbound
 * socket. This can lead to a reference count mishandling.
 */
static void test_seqpacket_transport_uaf_client(
    const struct test_opts *opts
) {
    int sockets[MAX_PORT_RETRIES];
    struct sockaddr_vm addr;
    int s, i, alen;

    s = vsock_bind(
        VMADDR_CID_LOCAL,
        VMADDR_PORT_ANY,
        SOCK_SEQPACKET
    );

    alen = sizeof(addr);
    if (getsockname(s, (struct sockaddr *)&addr, &alen)) {
        perror("getsockname");
        exit(EXIT_FAILURE);
    }

    for (i = 0; i < MAX_PORT_RETRIES; ++i) {
        sockets[i] = vsock_bind(
            VMADDR_CID_ANY,
            ++addr.svm_port,
            SOCK_SEQPACKET
        );
    }

    close(s);
    s = socket(AF_VSOCK, SOCK_SEQPACKET, 0);
    if (s < 0) {
        perror("socket");
        exit(EXIT_FAILURE);
    }

    if (!connect(s, (struct sockaddr *)&addr, alen)) {
        fprintf(stderr, "Unexpected connect() #1 success\n");
        exit(EXIT_FAILURE);
    }
    /* connect() #1 failed: transport set, sk in unbound list. */

    addr.svm_cid = VMADDR_CID_NONEXISTING;
```

```
48      if (!connect(s, (struct sockaddr *)&addr, alen)) {
49          fprintf(stderr, "Unexpected connect() #2 success\n");
50          exit(EXIT_FAILURE);
51      }
52      /* connect() #2 failed: transport unset, sk ref dropped? */
53
54      addr.svm_cid = VMADDR_CID_LOCAL;
55      addr.svm_port = VMADDR_PORT_ANY;
56
57      /* Vulnerable system may crash now. */
58      bind(s, (struct sockaddr *)&addr, alen);
59
60      close(s);
61      while (i--)
62          close(sockets[i]);
63
64      control_writeln("DONE");
65  }
```

The test case in question does not lead to a privilege escalation, but causes the kernel to crash on vulnerable systems. By examining it step by step, it is possible to understand what is going wrong in the reference counting mechanism associated with a vsock socket.

Step-by-step analysis

Let's look at every relevant step in the test case. Every involved function will be described in detail in section 3.4.

```
14      s = vsock_bind(
15          VMADDR_CID_LOCAL,
16          VMADDR_PORT_ANY,
17          SOCK_SEQPACKET
18      );
```

After initially declaring the variables, a new local (identified by CID = 1) socket is bound. By doing this, the integer variable `s` will contain a new valid port assigned from the kernel.

```
21      if (getsockname(s, (struct sockaddr *)&addr, &alen)) {
22          perror("getsockname");
23          exit(EXIT_FAILURE);
24      }
```

The instruction inside the `if` condition puts into `addr` the address of the new-bound socket. It terminates with a failure if `getsockname` returns `-1` (negative result) while it continues if it returns `0` (positive result).

```
26      for (i = 0; i < MAX_PORT_RETRIES; ++i) {
27          sockets[i] = vsock_bind(
28              VMADDR_CID_ANY,
29              ++addr.svm_port,
30              SOCK_SEQPACKET
31          );
32      }
```

The `for` loop arbitrarily binds 24 ports (unlike the instruction in row 14, where it is automatically assigned by the kernel), starting with the port immediately following the one assigned by the kernel, in order to occupy them.

```
34      close(s);
```

The first socket is then closed. Its only purpose was to automatically obtain the first port from the kernel.

```
35      s = socket(AF_VSOCK, SOCK_SEQPACKET, 0);
36      if (s < 0) {
37          perror("socket");
38          exit(EXIT_FAILURE);
39      }
40
41      if (!connect(s, (struct sockaddr *)&addr, alen)) {
42          fprintf(stderr, "Unexpected connect() #1 success\n");
43          exit(EXIT_FAILURE);
44      }
```

In this step, the "victim" socket for the exploit is created. `s` is reinitialized with no port, no peer, no bind, and no transport.

We want the `connect` to fail in a specific way. With this connection attempt, the kernel prepares the socket, selects the transport and attempts to assign it a local port. The 24 ports we set up earlier are used to steer the process along this error path so that the transport is set but the socket remains in the unbound list.

```
47      addr.svm_cid = VMADDR_CID_NONEXISTING;
48      if (!connect(s, (struct sockaddr *)&addr, alen)) {
49          fprintf(stderr, "Unexpected connect() #2 success\n");
50          exit(EXIT_FAILURE);
51      }
52      /* connect() #2 failed: transport unset, sk ref dropped? */
```

The CID is then set to a non-existent type to force the next connection attempt to fail; this one proceeds just like the previous one, and we again expect it to fail, but for a different reason . This causes the trasport to be unset and, at the same time, the reference count to be decreased even if the object is still existing.

```
54      addr.svm_cid = VMADDR_CID_LOCAL;
55      addr.svm_port = VMADDR_PORT_ANY;
56
```

```
57      /* Vulnerable system may crash now. */
58      bind(s, (struct sockaddr *)&addr, alen);
59
60      close(s);
61      while (i--)
62          close(sockets[i]);
63
64      control_writeln("DONE");
```

The final step demonstrates how the vulnerability was triggered, showing that, on a vulnerable system, attempting to bind a socket to an invalid object causes a crash.

3.4 Cause and functions involved

In order to understand exactly why the bug occurs, we can start by looking at the patch the developers used to fix it. Before the patch (kernel versions until 6.6.78), the vulnerable code looked like this:

```
1       void vsock_remove_sock(struct vsock_sock *vsk)
2       {
3           vsock_remove_bound(vsk);
4           vsock_remove_connected(vsk);
5       }
```

while in the version after the patch (6.6.79) it looked like this:

```
1       void vsock_remove_sock(struct vsock_sock *vsk)
2       {
3               /* Transport reassignment must not remove the binding. */
4               if (sock_flag(sk_vsock(vsk), SOCK_DEAD))
5                       vsock_remove_bound(vsk);
6
7               vsock_remove_connected(vsk);
8       }
```

The maintainers added a simple `if` that checks if the socket is really "dead" before removing the binding. We can better understand how it works by working backward, starting with the call to the `vsock_remove_bound` function.

Before examining the function itself, we need to explain how the **bound tables** approach works. There are two global (as all sockets on the system are placed within them) tables, one for **bound sockets** (sockets that have specified an address that they are responsible for) and one for **connected sockets** (sockets that have established a connection with another socket). The bound table contains an extra entry for a list of **unbound sockets**. These tables are managed as *intrusive lists* and are contained directly within the socket structs.

In the test case, the newly created `SOCK_SEQPACKET` socket is initially inserted into the unbound list, since no local address has been assigned to it yet. The

first `connect()` attempt implicitly tries to bind the socket, but the previously allocated consecutive ports force this operation to fail. As a result, the socket remains in the unbound list, although a transport has already been assigned to it.

The second `connect()` changes the destination so that a different transport selection is required. During this reassignment, the previous transport is released and the socket is removed through the normal bound-socket removal path, but the socket was never actually moved from the unbound list to one of the regular bound-table buckets. The vulnerable code therefore treats an unbound socket as if it were normally bound and drops a reference that should have been preserved. The subsequent `bind()` then operates on a socket whose reference count may already be incorrect, exposing the use-after-free condition.

Now that we've covered how bound tables work, we can examine the contents of `vsock_remove_bound`. Since the original function calls a wrapper that contains mutual exclusion instructions and the entire function that contains the actual logic, for simplicity we'll focus directly on the latter.

```
1       static void __vsock_remove_bound(struct vsock_sock *vsk)
2       {
3               list_del_init(&vsk->bound_table);
4               sock_put(&vsk->sk);
5       }
```

3.5 Overview of the exploit

Chapter 4

Setup and tests with UAFX

Chapter 5

Setup and tests with CountDown

Chapter 6

Conclusions

Bibliography

[1] Linux Market Share Statistics 2026: Desktop, Server and Enterprise Adoption Trends https://fosspost.org/linux-market-share-statistics/, 2026.

[2] Statically Discover Cross-Entry Use-After-Free Vulnerabilities in the Linux Kernel https://www.ndss-symposium.org/wp-content/uploads/2025-559-paper.pdf, 2025

[3] CountDown: Refcount-guided Fuzzing for Exposing Temporal Memory Errors in Linux Kernel https://dl.acm.org/doi/epdf/10.1145/3658644.3690320, 2024

[4] Infer: An Automatic Program Verifier for Memory Safety of C Programs https://scispace.com/pdf/infer-an-automatic-program-verifier-for-memory-safety-of-c-1d048xu12f.pdf, 2011

[5] Clang Static Analyzer https://github.com/llvm/llvm-project/blob/main/clang/lib/StaticAnalyzer/README.txt, 2008

[6] Syzkaller: unsupervised coverage-guided kernel fuzzer https://github.com/google/syzkaller/blob/master/docs/internals.md, 2015

[7] GNU Compiler Collection (GCC) https://github.com/gcc-mirror/gcc, 1987

[8] Spatio-Temporal Context Reduction: A Pointer-Analysis-BasedStatic Approach for Detecting Use-After-Free Vulnerabilities https://dl.acm.org/doi/epdf/10.1145/3180155.3180178, 2018

[9] Cppcheck: static analysis of C/C++ code https://github.com/cppcheck-opensource/cppcheck, 2007

[10] SVF: Static Value-Flow Analysis Framework for Source Code https://github.com/SVF-tools/SVF, 2016

[11] FreeWill: Automatically Diagnosing Use-after-free Bugs via Reference Miscounting Detection on Binaries https://www.usenix.org/system/files/sec22-he-liang.pdf, 2022

[12] LinKRID: Vetting Imbalance Reference Counting in Linux kernel with Symbolic Execution https://www.usenix.org/system/files/sec22-liu-jian.pdf, 2022

[13] CRCount: Pointer Invalidation with Reference Counting to Mitigate Use-after-free in Legacy C/C++ https://www.ndss-symposium.org/wp-content/uploads/2019/02/ndss2019_05A-4_Shin_paper.pdf, 2019

[14] UAFChecker: Scalable Static Detection ofUse-After-Free Vulnerabilities https://dl.acm.org/doi/epdf/10.1145/2660267.2662394, 2014

[15] Effective Static Analysis of Concurrency Use-After-Free Bugs in Linux Device Drivers https://www.usenix.org/system/files/atc19-bai.pdf, 2019

[16] SinkFinder: Harvesting Hundreds of Unknown InterestingFunction Pairs with Just One Seed https://dl.acm.org/doi/epdf/10.1145/3368089.3409678, 2020

[17] Goshawk: Hunting Memory Corruptions via Structure-Aware and Object-Centric Memory Operation Synopsis https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9833613, 2022

[18] Detecting Kernel Refcount Bugs with Two-Dimensional Consistency Checking https://yuanxzhang.github.io/paper/cid-security21.pdf, 2021

[19] The GNU Allocator https://sourceware.org/glibc/manual/2.27/html_node/The-GNU-Allocator.html

[20] Efficiency Considerations for malloc https://sourceware.org/glibc/manual/2.23/html_node/Efficiency-and-Malloc.html

[21] af_vsock module, Linux kernel code, v6.6.78 https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/tree/net/vmw_vsock/af_vsock.c?h=v6.6.78, 2025